

Web Page Phishing Detection Dataset - EDA

```
In [1]: import os
import kagglehub
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [ ]: path = kagglehub.dataset_download("shashwatwork/web-page-phishing-detection-dataset")
csv_file = os.path.join(path, "dataset_phishing.csv")
phishing_df = pd.read_csv(csv_file)
```

```
In [3]: # Preview the dataset
phishing_df.head(5)
```

```
Out[3]:
```

	url	length_url	length_hostname	ip	nb_dots
0	http://www.crestonwood.com/router.php	37	19	0	3
1	http://shadetreetechnology.com/V4/validation/a...	77	23	1	1
2	https://support-appleld.com.secureupdate.duila...	126	50	1	4
3	http://rgipt.ac.in	18	11	0	2
4	http://www.iracing.com/tracks/gateway-motorspo...	55	15	0	2

5 rows × 89 columns



```
In [4]: phishing_df.info() # different data types found in dataset
```

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 11430 entries, 0 to 11429

Data columns (total 89 columns):

#	Column	Non-Null Count	Dtype
0	url	11430 non-null	object
1	length_url	11430 non-null	int64
2	length_hostname	11430 non-null	int64
3	ip	11430 non-null	int64
4	nb_dots	11430 non-null	int64
5	nb_hyphens	11430 non-null	int64
6	nb_at	11430 non-null	int64
7	nb_qm	11430 non-null	int64
8	nb_and	11430 non-null	int64
9	nb_or	11430 non-null	int64
10	nb_eq	11430 non-null	int64
11	nb_underscore	11430 non-null	int64
12	nb_tilde	11430 non-null	int64
13	nb_percent	11430 non-null	int64
14	nb_slash	11430 non-null	int64
15	nb_star	11430 non-null	int64
16	nb_colon	11430 non-null	int64
17	nb_comma	11430 non-null	int64
18	nb_semicolumn	11430 non-null	int64
19	nb_dollar	11430 non-null	int64
20	nb_space	11430 non-null	int64
21	nb_www	11430 non-null	int64
22	nb_com	11430 non-null	int64
23	nb_dslash	11430 non-null	int64
24	http_in_path	11430 non-null	int64
25	https_token	11430 non-null	int64
26	ratio_digits_url	11430 non-null	float64
27	ratio_digits_host	11430 non-null	float64
28	punycode	11430 non-null	int64
29	port	11430 non-null	int64
30	tld_in_path	11430 non-null	int64
31	tld_in_subdomain	11430 non-null	int64
32	abnormal_subdomain	11430 non-null	int64
33	nb_subdomains	11430 non-null	int64
34	prefix_suffix	11430 non-null	int64
35	random_domain	11430 non-null	int64
36	shortening_service	11430 non-null	int64
37	path_extension	11430 non-null	int64
38	nb_redirection	11430 non-null	int64
39	nb_external_redirection	11430 non-null	int64
40	length_words_raw	11430 non-null	int64
41	char_repeat	11430 non-null	int64
42	shortest_words_raw	11430 non-null	int64
43	shortest_word_host	11430 non-null	int64
44	shortest_word_path	11430 non-null	int64
45	longest_words_raw	11430 non-null	int64
46	longest_word_host	11430 non-null	int64
47	longest_word_path	11430 non-null	int64
48	avg_words_raw	11430 non-null	float64
49	avg_word_host	11430 non-null	float64
50	avg_word_path	11430 non-null	float64

51	phish_hints	11430	non-null	int64
52	domain_in_brand	11430	non-null	int64
53	brand_in_subdomain	11430	non-null	int64
54	brand_in_path	11430	non-null	int64
55	suspicious_tld	11430	non-null	int64
56	statistical_report	11430	non-null	int64
57	nb_hyperlinks	11430	non-null	int64
58	ratio_intHyperlinks	11430	non-null	float64
59	ratio_extHyperlinks	11430	non-null	float64
60	ratio_nullHyperlinks	11430	non-null	int64
61	nb_extCSS	11430	non-null	int64
62	ratio_intRedirection	11430	non-null	int64
63	ratio_extRedirection	11430	non-null	float64
64	ratio_intErrors	11430	non-null	int64
65	ratio_extErrors	11430	non-null	float64
66	login_form	11430	non-null	int64
67	external_favicon	11430	non-null	int64
68	links_in_tags	11430	non-null	float64
69	submit_email	11430	non-null	int64
70	ratio_intMedia	11430	non-null	float64
71	ratio_extMedia	11430	non-null	float64
72	sfh	11430	non-null	int64
73	iframe	11430	non-null	int64
74	popup_window	11430	non-null	int64
75	safe_anchor	11430	non-null	float64
76	onmouseover	11430	non-null	int64
77	right_clic	11430	non-null	int64
78	empty_title	11430	non-null	int64
79	domain_in_title	11430	non-null	int64
80	domain_with_copyright	11430	non-null	int64
81	whois_registered_domain	11430	non-null	int64
82	domain_registration_length	11430	non-null	int64
83	domain_age	11430	non-null	int64
84	web_traffic	11430	non-null	int64
85	dns_record	11430	non-null	int64
86	google_index	11430	non-null	int64
87	page_rank	11430	non-null	int64
88	status	11430	non-null	object

dtypes: float64(13), int64(74), object(2)

memory usage: 7.8+ MB

In [5]: phishing_df.describe()

```
Out[5]:
```

	length_url	length_hostname	ip	nb_dots	nb_hyphens	n
count	11430.000000	11430.000000	11430.000000	11430.000000	11430.000000	11430.000000
mean	61.126684	21.090289	0.150569	2.480752	0.997550	0.020000
std	55.297318	10.777171	0.357644	1.369686	2.087087	0.150000
min	12.000000	4.000000	0.000000	1.000000	0.000000	0.000000
25%	33.000000	15.000000	0.000000	2.000000	0.000000	0.000000
50%	47.000000	19.000000	0.000000	2.000000	0.000000	0.000000
75%	71.000000	24.000000	0.000000	3.000000	1.000000	0.000000
max	1641.000000	214.000000	1.000000	24.000000	43.000000	4.000000

8 rows × 87 columns



```
In [6]: print(phishing_df.duplicated().any()) # Find out if there are any duplicate rows
```

False

This dataset does not contain any duplicate rows.

```
In [7]: # identify any other inconsistent data in dataframe
phishing_df.dtypes
```

```
Out[7]: url                object
length_url              int64
length_hostname         int64
ip                      int64
nb_dots                 int64
...
web_traffic             int64
dns_record              int64
google_index            int64
page_rank               int64
status                  object
Length: 89, dtype: object
```

```
In [8]: # I decided to drop the 'url' column because it is not needed for this analysis
phishing_df = phishing_df.drop('url', axis=1)
```

```
In [10]: # identify inconsistent data by checking for missing or null values
phishing_df.isnull().any().any()
```

```
Out[10]: np.False_
```

These results show that there are no missing values in the dataset.

```
In [11]: # encode categorical variables
categorical = phishing_df.select_dtypes(exclude=['number']).columns.tolist()
```

categorical

Out[11]: ['status']

Conclusions:

This data is usable after modifying the data to remove the unnecessary columns. In this case, only the 'url' column was removed. I chose to remove it because the other columns already provide useful information about the url so the url name itself is unnecessary. The url name is also unique so it would not make a good feature to analyze. There is no class imbalance, as the categorical variable (status) which is the target, only has 2 items (0 for legitimate and 1 for phishing).

```
In [ ]: phishing_df['status'] = phishing_df['status'].replace({'legitimate': 0, 'phishing': 1})
```

```
In [13]: # print correlation matrix for all features
numerical_df = phishing_df.select_dtypes(include=['float64', 'int64'])
corr_matrix = numerical_df.corr()
corr_matrix
```

Out[13]:

	length_url	length_hostname	ip	nb_dots	nb_hyphens	nb_at
length_url	1.000000	0.223025	0.453961	0.443589	0.399564	0.150735
length_hostname	0.223025	1.000000	0.252013	0.408956	0.057702	0.071793
ip	0.453961	0.252013	1.000000	0.288398	0.109860	0.059401
nb_dots	0.443589	0.408956	0.288398	1.000000	0.045099	0.263283
nb_hyphens	0.399564	0.057702	0.109860	0.045099	1.000000	0.018770
...	...	...	...	...	...	...
web_traffic	0.072205	0.163238	0.167930	0.087969	-0.041464	-0.009455
dns_record	0.023357	-0.023344	0.127823	0.126659	-0.031477	0.031611
google_index	0.236395	0.213990	0.270743	0.209616	-0.018828	0.113217
page_rank	-0.102582	-0.159342	-0.218968	-0.097312	0.104341	-0.066356
status	0.248580	0.238322	0.321698	0.207029	-0.100107	0.142915

88 rows × 88 columns



```
In [ ]: # Set the threshold value for high correlation where |r| > 0.75 to filter out some
threshold = 0.75
corr_matrix = phishing_df.corr()
corr_pairs = corr_matrix.unstack()
high_corr_pairs = corr_pairs[abs(corr_pairs) > threshold]
high_corr_pairs = high_corr_pairs[high_corr_pairs != 1]
high_corr_pairs = high_corr_pairs[high_corr_pairs.index.get_level_values(0) < high_
```

```
print(high_corr_pairs)
```

length_url	length_words_raw	0.794710
ip	ratio_digits_url	0.768906
nb_and	nb_eq	0.906404
longest_word_path	longest_words_raw	0.968503
avg_words_raw	longest_words_raw	0.784753
avg_word_host	shortest_word_host	0.800014
	longest_word_host	0.816313
avg_word_path	longest_word_path	0.791198
links_in_tags	ratio_inHyperlinks	0.799374

dtype: float64

```
In [15]: print(len(high_corr_pairs))
```

9

There are a lot of highly correlated pairs, suggesting some redundancy in the dataset.

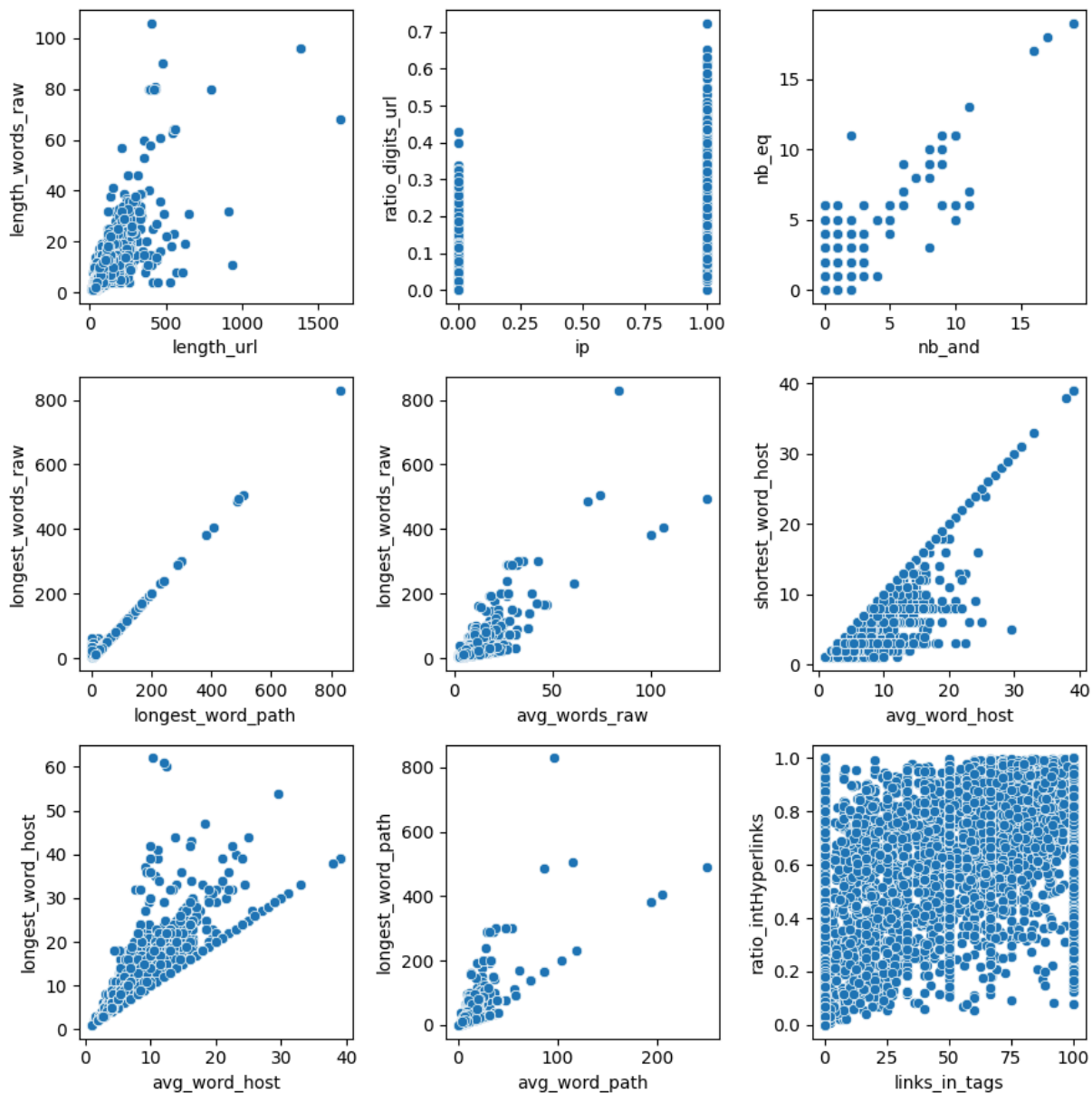
```
In [16]: # plot scatterplots of the pairs with large correlation where |r| > 0.75
n_cols = 3
n_rows = 3

fig, axes = plt.subplots(nrows=n_rows, ncols=n_cols, figsize=(n_cols * 3, n_rows *
axes = axes.flatten() # Flatten the axes to easily loop through

for i, (pair, ax) in enumerate(zip(high_corr_pairs.index, axes)):
    col1, col2 = pair
    sns.scatterplot(x=phishing_df[col1], y=phishing_df[col2], ax=ax)
    ax.set_xlabel(col1)
    ax.set_ylabel(col2)

for j in range(i + 1, len(axes)):
    axes[j].axis('off')

plt.tight_layout()
plt.show()
```



```
In [17]: # Compute the correlation matrix where correlation ( $|r| > 0.3$ )
corr_matrix = phishing_df.corr()
status_corr = corr_matrix['status']
filtered_status_corr = status_corr[abs(status_corr) > 0.3]
print(filtered_status_corr)

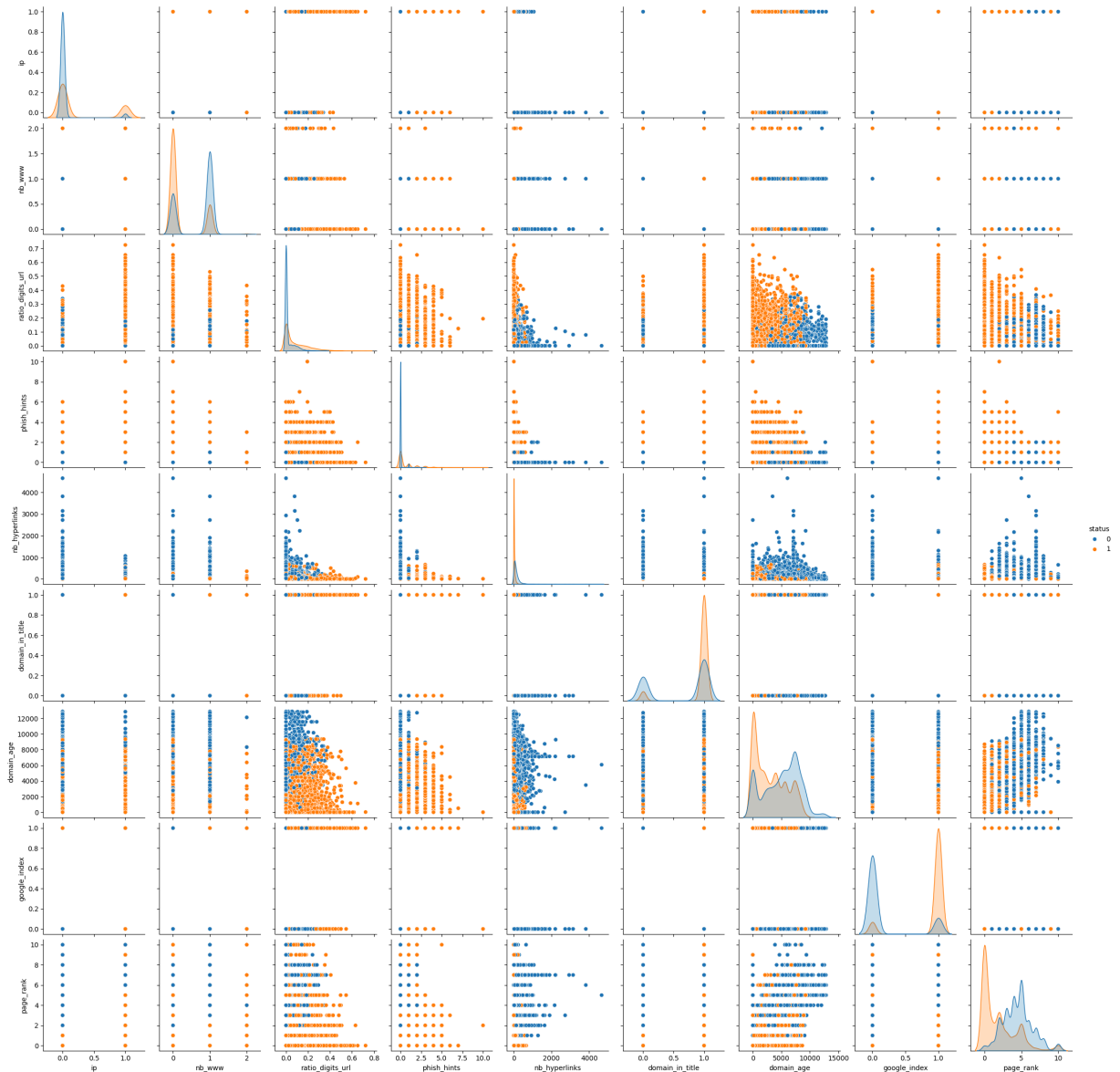
# Create the pairplot for highly correlated columns, with 'status' as the hue
sns.pairplot(phishing_df[filtered_status_corr.index], hue='status')

# Show the plot
plt.show()
```

```

ip                0.321698
nb_www            -0.443468
ratio_digits_url  0.356395
phish_hints       0.335393
nb_hyperlinks     -0.342628
domain_in_title   0.342807
domain_age        -0.331889
google_index      0.731171
page_rank         -0.511137
status            1.000000
Name: status, dtype: float64

```



Although some of the features show a high correlation to each other, none of them in this dataset have a high correlation with the target, suggesting that a linear model would not be ideal for this dataset. There are several outliers which can be seen in the scatterplots and pair wise plots. There are items that are highly correlated with each other, which may be redundant. For example, 'avg_word_path' and 'longest_word_path'. I did not graph line plots for this dataset because there are no date type objects. 'IP', 'nb_www', 'ratio_digits_url', 'phish_hints', 'nb_hyperlinks', 'domain_in_title', 'domain_age', 'google_index', 'page_rank', all

had correlations greater than 0.3 with the status column. Therefore, these are the features we'd want to use when fitting our model.

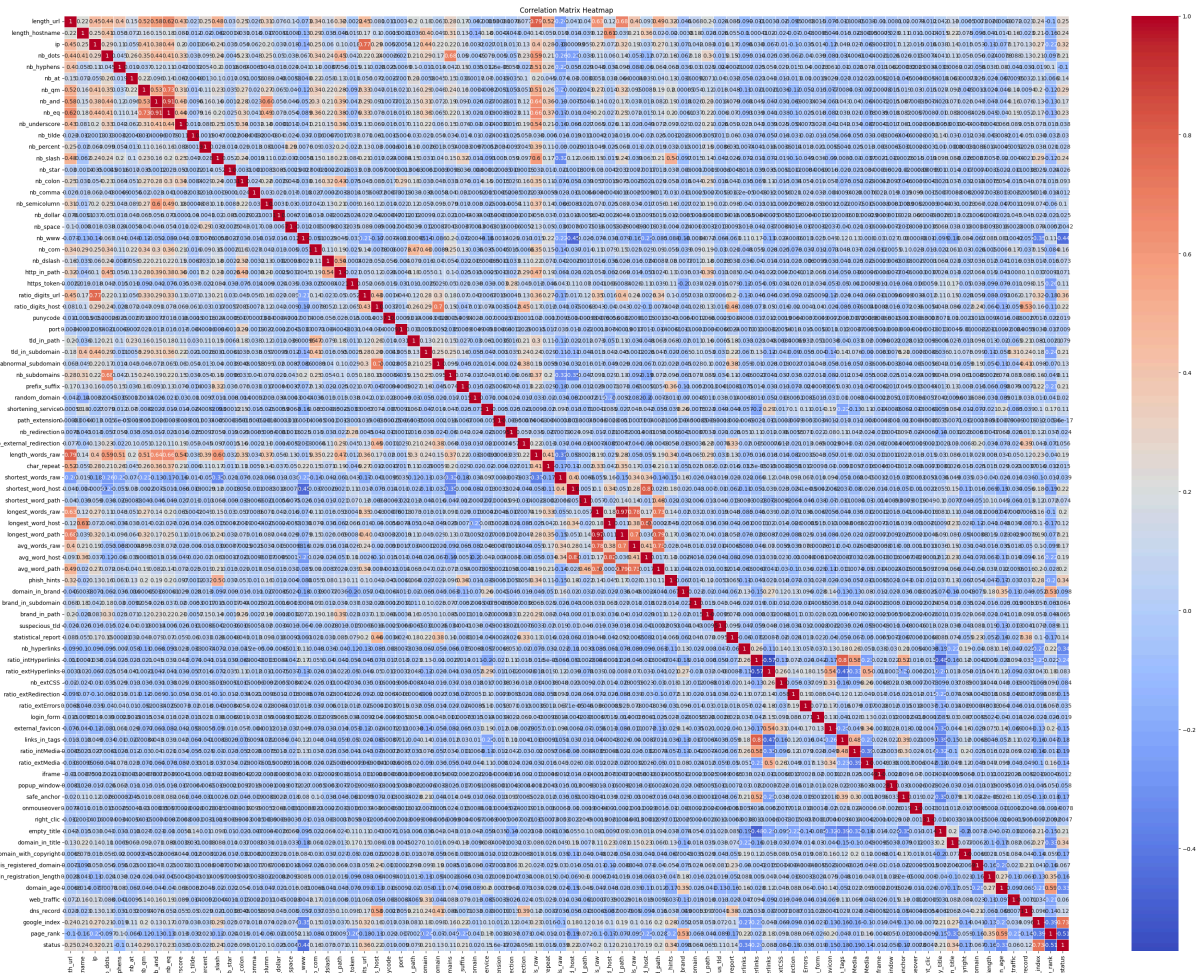
```
In [18]: cols_to_drop = phishing_df.columns[phishing_df.nunique() == 1]
phishing_df = phishing_df.drop(cols_to_drop, axis=1)
```

```
In [ ]: # print correlation matrix for all features to target variable 'status'
correlation_matrix = phishing_df.corr()
correlations_with_target = correlation_matrix['status']

sorted_correlations_abs = correlations_with_target.abs().sort_values(ascending=False)
print(sorted_correlations_abs)
```

```
status          1.000000e+00
google_index     7.311708e-01
page_rank        5.111371e-01
nb_www           4.434677e-01
ratio_digits_url 3.563946e-01
...
port             9.011116e-03
onmouseover      7.787061e-03
right_click      4.680056e-03
nb_space         4.193222e-03
path_extension   5.592660e-17
Name: status, Length: 82, dtype: float64
```

```
In [ ]: # create heatmap for correlation matrix of all features
plt.figure(figsize=(40, 30))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', linewidths=0.5)
plt.title('Correlation Matrix Heatmap')
plt.show()
```



```
In [ ]: # Compute the correlation matrix where correlation (|r| > 0.25) to filter out some
filtered_status_corr = correlations_with_target[sorted_correlations_abs > 0.25]
print(filtered_status_corr)
```

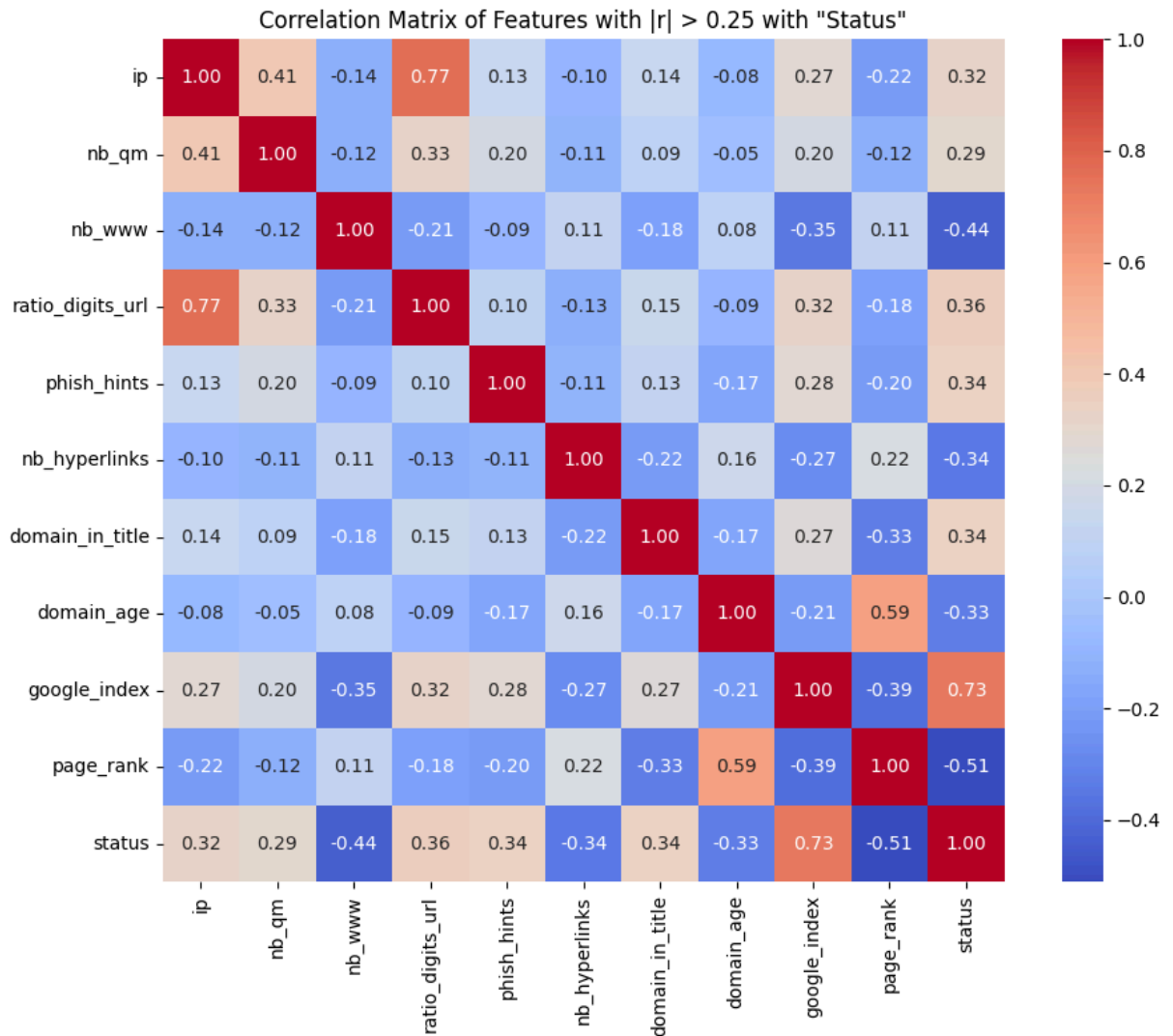
```
ip          0.321698
nb_qm       0.294319
nb_www      -0.443468
ratio_digits_url 0.356395
phish_hints 0.335393
nb_hyperlinks -0.342628
domain_in_title 0.342807
domain_age  -0.331889
google_index 0.731171
page_rank   -0.511137
status      1.000000
Name: status, dtype: float64
```

```
In [23]: # create heatmap
selected_features = filtered_status_corr.index.tolist()

# Create a correlation matrix
corr_matrix = phishing_df[selected_features].corr()

# Plot the heatmap
```

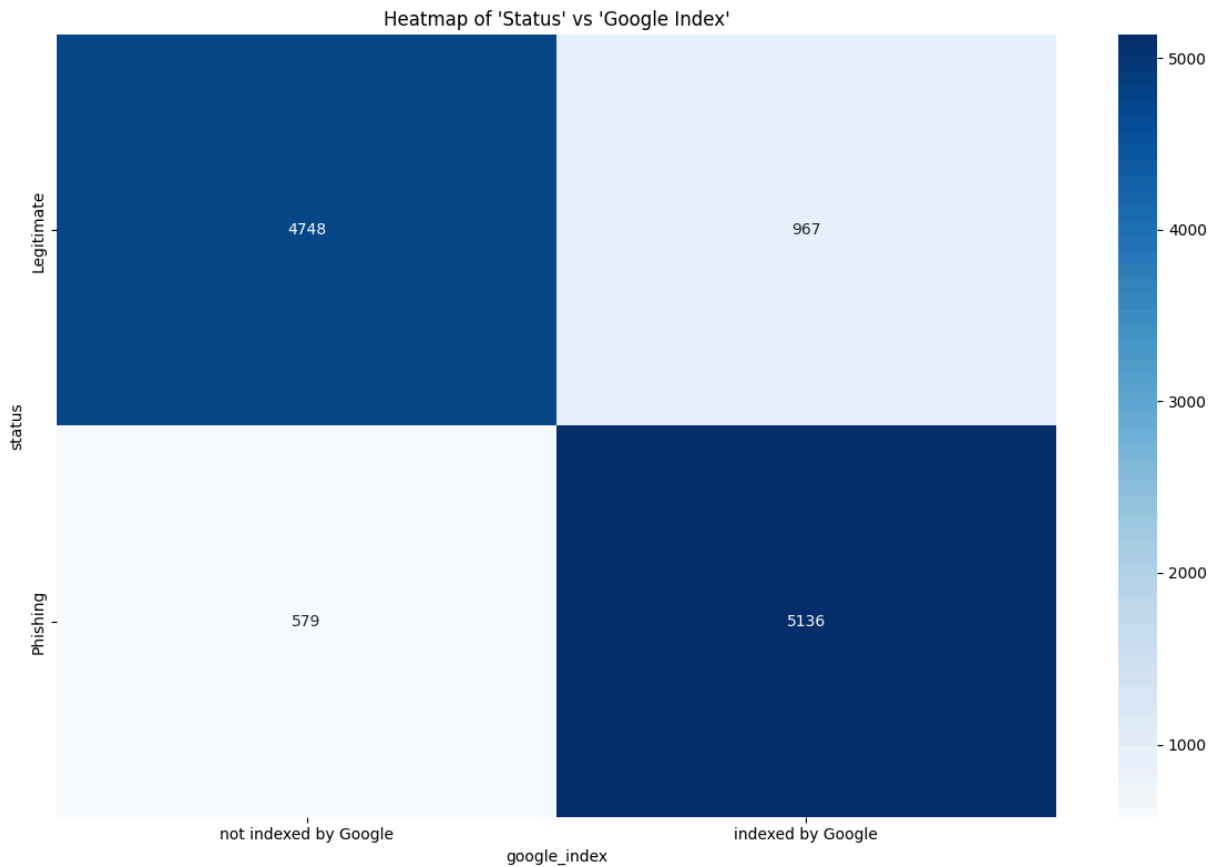
```
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", square=True)
plt.title('Correlation Matrix of Features with |r| > 0.25 with "Status"')
plt.tight_layout()
plt.show()
```



```
In [ ]: heatmap_data = pd.crosstab(phishing_df['status'], phishing_df['google_index'])

# Plot heatmap
plt.figure(figsize=(12, 8))
ax = sns.heatmap(heatmap_data, cmap="Blues", annot=True, fmt="d")

plt.title("Heatmap of 'Status' vs 'Google Index'")
ax.set_xticklabels(['not indexed by Google', 'indexed by Google'])
ax.set_yticklabels(['Legitimate', 'Phishing'])
plt.tight_layout()
plt.show()
```



```
In [25]: heatmap_data = pd.crosstab(phishing_df['status'], phishing_df['page_rank'])

# Plot the heatmap
plt.figure(figsize=(12, 8))
ax = sns.heatmap(heatmap_data, cmap="Blues", annot=True, fmt="d")
ax.set_yticklabels(['Legitimate', 'Phishing'])
plt.title("Heatmap of 'Status' vs 'Page Rank'")

plt.tight_layout()
plt.show()
```

